

TechEdu Insight — 성능 개선 계획

Lighthouse 분석 기준 (시크릿 창): Performance **68**점, LCP **11.5s**, Speed Index **7.0s**
목표: Performance **90**점 이상, LCP **2.5s** 이하

현재 상태 요약

지표	현재	목표
Performance	68	90+
LCP	11.5s	< 2.5s
Speed Index	7.0s	< 3.0s
FCP	1.0s	유지
TBT	20ms	유지

핵심 원인: Supabase Storage 이미지를 원본 크기(1721×968px)로 전송하면서 표시 크기(378×23px)로 렌더링 → LCP Element render delay 3,080ms 발생

Phase 1 — 이미지 최적화 (최우선, 예상 효과 가장 큼)

1-1. `next/image` 컴포넌트로 전환

문제: `<img>` 태그로 Supabase 이미지를 직접 불러오고 있음. 원본 크기 그대로 전송됨.

수정 방법:

tsx

```
// Before


// After
import Image from 'next/image';

<Image
  src="https://xxx.supabase.co/storage/v1/object/..."
  fill
  sizes="(max-width: 768px) 100vw, 378px"
  className="object-cover object-center transition-transform duration-800 group-hove
  alt="프로젝트 썸네일"
/>
```

next.config.js 도메인 허용 추가 필요:

```
js
// next.config.js
module.exports = {
  images: {
    remotePatterns: [
      {
        protocol: 'https',
        hostname: '*.supabase.co',
        pathname: '/storage/v1/object/**',
      },
    ],
  },
};
```

체크리스트:

- ☐ 프로젝트 카드 썸네일 이미지 교체
- ☐ 히어로/배너 이미지가 있다면 `priority={true}` 추가
- ☐ `next.config.js`에 `remotePatterns` 추가
- ☐ 부모 컨테이너에 `position: relative` 확인 (`fill` 사용 시 필수)

1-2. Supabase Transform API로 서버 사이드 리사이즈

문제: 원본 이미지(1721×968)를 그대로 전송 후 CSS로만 축소 표시 중.

수정 방법:

```
ts

// utils/supabase-image.ts 생성
export function getOptimizedImageUrl(
  path: string,
  width: number,
  height: number
): string {
  const supabase = createClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  );

  const { data } = supabase.storage
    .from('your-bucket-name')
    .getPublicUrl(path, {
      transform: {
        width,
        height,
        resize: 'cover',
      },
    });

  return data.publicUrl;
}

// 사용 예시
const imageUrl = getOptimizedImageUrl(imagePath, 400, 300);
```

체크리스트:

- ☐ 유틸 함수 생성
- ☐ 프로젝트 카드에서 최적화된 URL 사용
- ☐ 표시되는 곳마다 적절한 width/height 값 설정

1-3. Supabase Storage 캐시 TTL 연장

문제: 현재 이미지 Cache-Control이 (1h)(3600초). 재방문 시에도 매번 재다운로드됨.

수정 방법:

```
ts
```

```
// 이미지 업로드 시 cacheControl 설정
await supabase.storage
  .from('your-bucket')
  .upload(path, file, {
    cacheControl: '31536000', // 1년 (초 단위)
    upsert: false,
  });
```

기존에 업로드된 이미지는 Supabase 대시보드 → Storage → 버킷 설정에서 변경하거나, 재업로드 필요.

체크리스트:

- ☐ 신규 업로드 코드에 `cacheControl: '31536000'` 추가
- ☐ 기존 이미지 캐시 설정 확인

Phase 2 — JavaScript 번들 최적화

2-1. 대형 청크 코드 분할 (Dynamic Import)

문제: `@cnakm9u.65g2.js` (59 KiB), `@bogtdbh.dcu1.js` (70 KiB) — 사용하지 않는 코드 포함. 총 76 KiB 절감 가능.

수정 방법:

```
tsx

import dynamic from 'next/dynamic';

// 초기 렌더에 불필요한 무거운 컴포넌트
const HeavyChart = dynamic(() => import('@components/HeavyChart'), {
  loading: () => <div>로딩 중...</div>,
  ssr: false,
});

const HeavyEditor = dynamic(() => import('@components/HeavyEditor'), {
  ssr: false,
});

// 이벤트 발생 시에만 필요한 라이브러리
async function handleExport() {
  const { exportToPDF } = await import('@lib/pdf-export');
  exportToPDF();
}
```

번들 분석으로 원인 파악:

```
bash
```

```
npm install @next/bundle-analyzer --save-dev
```

```
js
```

```
// next.config.js
const withBundleAnalyzer = require('@next/bundle-analyzer')({
  enabled: process.env.ANALYZE === 'true',
});

module.exports = withBundleAnalyzer({
  // 기존 설정
});
```

```
bash
```

```
ANALYZE=true npm run build
```

체크리스트:

- ☐ bundle analyzer 설치 및 실행
- ☐ 큰 청크의 원인 라이브러리 파악
- ☐ 초기 렌더에 불필요한 컴포넌트 `dynamic import`로 전환
- ☐ 이벤트 기반 기능은 필요 시점에 `await import()` 사용

2-2. Legacy JS 폴리필 제거

문제: `Array.prototype.at`, `Object.fromEntries`, `String.prototype.trimEnd` 등 현대 브라우저에 이미 내장된 메서드들의 폴리필이 번들에 포함됨 (14 KiB 낭비).

수정 방법:

```
json
```

```
// package.json에 추가
{
  "browserslist": [
    "last 2 Chrome versions",
    "last 2 Firefox versions",
    "last 2 Safari versions",
    "last 2 Edge versions"
  ]
}
```

```
js

// next.config.js
module.exports = {
  experimental: {
    browsersListForSwc: true,
  },
  // 기존 설정
};
```

체크리스트:

- ☐ `package.json`에 `browserslist` 추가
 - ☐ `next.config.js`에 `browsersListForSwc: true` 추가
 - ☐ 빌드 후 번들 크기 확인
-

Phase 3 — 3rd Party 스크립트 최적화

3-1. Next.js Script 컴포넌트로 로딩 전략 변경

문제: 외부 스크립트(Analytics 등)가 메인 콘텐츠 렌더링을 지연시킴.

수정 방법:

```
tsx

import Script from 'next/script';

// Before: <script src="..."> 직접 사용

// After: strategy로 로딩 시점 제어
<Script
  src="https://analytics-provider.com/script.js"
  strategy="lazyOnload" // 페이지 로드 완료 후 로드
/>

// strategy 옵션:
// - "beforeInteractive" : HTML 파싱 전 (SSR 필수 스크립트만)
// - "afterInteractive"  : 페이지 hydration 후 (기본값, GA 등)
// - "lazyOnload"       : 모든 리소스 로드 후 (채팅 위젯 등)
```

체크리스트:

- ☐ `_app.tsx` 또는 `layout.tsx`에서 외부 스크립트 찾기
 - ☐ `<Script strategy="lazyOnload">` 또는 `"afterInteractive"`로 교체
-

Phase 4 — SEO 수정

4-1. 검색엔진 차단 해제

문제: `Page is blocked from indexing` — 검색 크롤러가 페이지를 수집하지 못함. SEO 63점.

수정 방법:

```
tsx

// app/layout.tsx 또는 각 page.tsx에서 확인
export const metadata = {
  robots: {
    index: true,    // noindex → index로 변경
    follow: true,
  },
};
```

```
ts

// 또는 robots.txt 확인
// public/robots.txt
User-agent: *
Allow: /          # Disallow → Allow로 변경
```

체크리스트:

- ☐ `metadata.robots`에 `noindex` 설정 여부 확인
- ☐ `public/robots.txt` 내용 확인
- ☐ Vercel 환경변수에서 `X-Robots-Tag` 헤더 확인
- ☐ 수정 후 Google Search Console에서 색인 요청

수정 순서 및 예상 효과

순서	작업	난이도	예상 LCP 개선
1	<code>next/image</code> 로 전환 + <code>remotePatterns</code> 설정	낮음	11.5s → ~3s
2	Supabase Transform API 적용	낮음	~3s → ~2s
3	Dynamic import로 청크 분할	중간	변들 76 KiB 감소
4	Legacy 폴리필 제거	낮음	변들 14 KiB 감소
5	Script 로딩 전략 변경	낮음	체감 속도 개선

순서	작업	난이도	예상 LCP 개선
6	SEO noindex 해제	낮음	SEO 점수 개선

1번 작업만으로 Performance 점수가 68 → 85+ 수준으로 개선될 가능성이 높음

참고 문서

- [Next.js Image Optimization](#)
- [Supabase Storage Transform](#)
- [Next.js Bundle Analyzer](#)
- [Next.js Script Component](#)